

Descripción

Este proyecto consiste en una API REST hecha con Spring Boot para gestionar un catálogo de música. En ella se pueden guardar artistas, canciones y playlists, además de relacionarlas entre sí.

El objetivo del proyecto es practicar el uso de JPA, las relaciones entre entidades, consultas, validaciones y seguridad básica.

Diagrama ER

erDiagram

ARTISTAS ||--o{ CANCIONES : tiene
CANCIONES }o--o{ PLAYLISTS : aparece_en

ARTISTAS {
 BIGINT id PK
 VARCHAR nombre
 VARCHAR pais
 VARCHAR genero_musical
 INTEGER anio_debut
 BOOLEAN activo
}

CANCIONES {
 BIGINT id PK
 VARCHAR titulo
 VARCHAR album
 INTEGER duracion
 BIGINT artista_id FK

```
}  
PLAYLISTS {  
    BIGINT id PK  
    VARCHAR nombre  
    VARCHAR descripcion  
    BOOLEAN publica  
}  
  
PLAYLIST_CANCIONES {  
    BIGINT playlist_id FK  
    BIGINT cancion_id FK  
}
```

Lista de endpoints

Artistas

- GET /api/v1/artistas
- GET /api/v1/artistas/{id}
- POST /api/v1/artistas
- PUT /api/v1/artistas/{id}
- DELETE /api/v1/artistas/{id}
- GET /api/v1/artistas/buscar

Canciones

- GET /api/v1/canciones
- GET /api/v1/canciones/{id}
- POST /api/v1/canciones
- PUT /api/v1/canciones/{id}
- DELETE /api/v1/canciones/{id}

- GET /api/v1/canciones/buscar
- GET /api/v1/canciones/artista/{artistaId}
- GET /api/v1/canciones/artista/{artistaId}/contador

Playlists

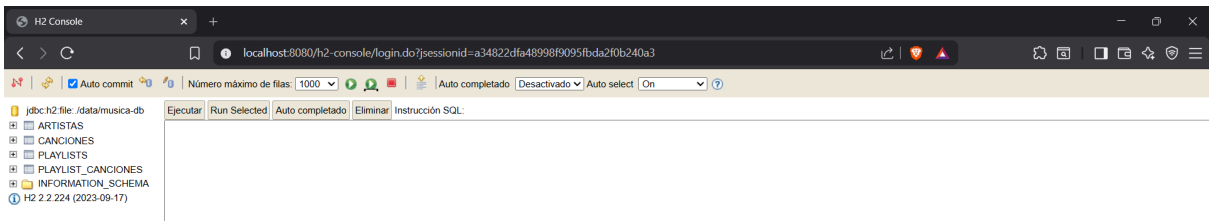
- GET /api/v1/playlists
 - GET /api/v1/playlists/{id}
 - POST /api/v1/playlists
 - PUT /api/v1/playlists/{id}
 - DELETE /api/v1/playlists/{id}
 - GET /api/v1/playlists/buscar
 - PUT /api/v1/playlists/{playlistId}/canciones/{cancionId}
 - GET /api/v1/playlists/artista/{artistaId}
-

Decisiones técnicas

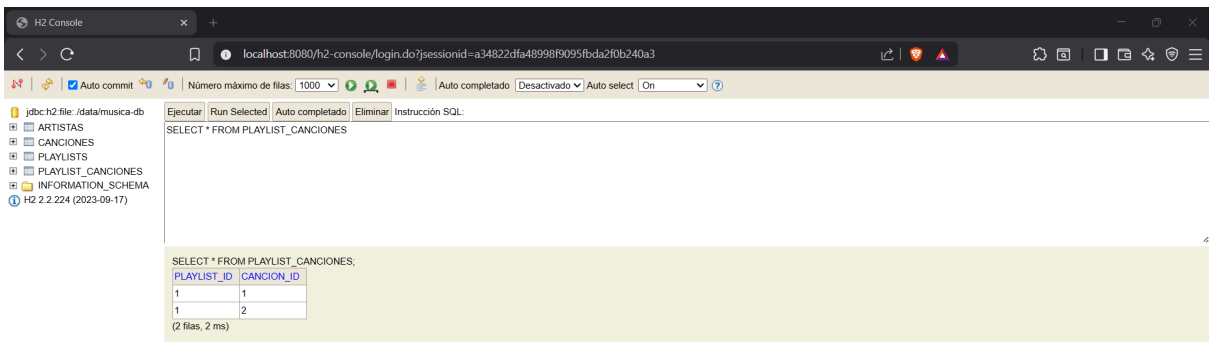
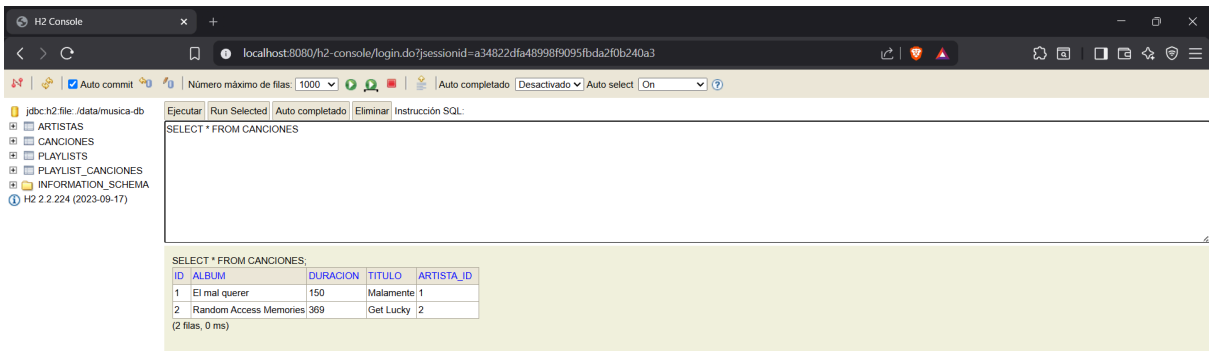
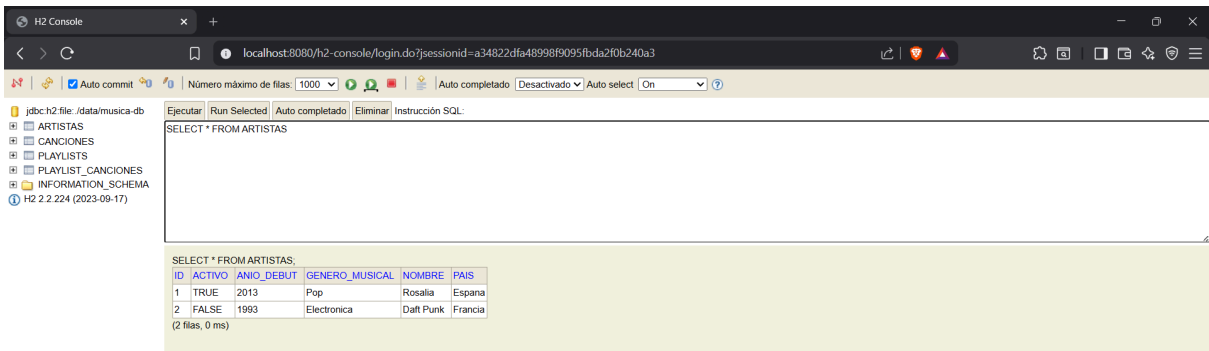
- Se ha utilizado Spring Boot para crear la API REST.
 - La base de datos utilizada es H2, almacenada en un fichero para que los datos no se borren al cerrar la aplicación.
 - Se ha seguido una estructura por capas (Controller, Service y Repository) para organizar mejor el código.
 - Se utilizan relaciones OneToMany, ManyToOne y ManyToMany entre las entidades.
 - Para las búsquedas se utilizan parámetros opcionales y ordenación mediante sortBy y order.
 - Algunas consultas se realizan con JPQL para practicar este tipo de consultas.
 - Se ha añadido Spring Security para que solo los métodos POST, PUT y DELETE necesiten autenticación.
 - También se han añadido validaciones para evitar introducir datos incorrectos.
-

Capturas de la base de datos

Tablas



Datos almacenados



SQL generado por Hibernate

```
Administrador: Windows PowerShell
2026-06-05T13:55:55.407+01:00 INFO 15564 --- [musica-api] [nio-8080-exec-1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2026-06-05T13:55:55.408+01:00 INFO 15564 --- [musica-api] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2026-06-05T13:55:55.412+01:00 INFO 15564 --- [musica-api] [nio-8080-exec-1] o.s.web.servlet.DispatcherServlet : Completed initialization in 1 ms

Hibernate:
select
  a1_0.id,
  a1_0.activo,
  a1_0.anio_debut,
  a1_0.genero_musical,
  a1_0.nombre,
  a1_0.pais
from
  artistas a1_0
Hibernate:
select
  c1_0.id,
  c1_0.album,
  c1_0.artista_id,
  c1_0.duracion,
  c1_0.titulo
from
  canciones c1_0
```

```
Administrador: Windows PowerShell

Hibernate:
select
  c1_0.id,
  c1_0.album,
  c1_0.artista_id,
  c1_0.duracion,
  c1_0.titulo
from
  canciones c1_0
Hibernate:
select
  a1_0.id,
  a1_0.activo,
  a1_0.anio_debut,
  a1_0.genero_musical,
  a1_0.nombre,
  a1_0.pais
from
  artistas a1_0
where
  a1_0.id=?
Hibernate:
select
  a1_0.id,
  a1_0.activo,
  a1_0.anio_debut,
  a1_0.genero_musical,
  a1_0.nombre,
  a1_0.pais
from
  artistas a1_0
where
  a1_0.id=?
```